

EasyClassifier analysis report

demo (Iris flowers)

10 October 2026 · EasyClassifier 0.8.2

Summary

The goal was to predict *Species* from 4 other columns, using 149 rows. 7 classifiers were compared and **Neural Network (MLP)** was selected. Its honest final accuracy was **95.30%** (guessing at random would give about 33% with equally sized classes).

1 Data and question

File: demo (Iris flowers); 150 rows and 5 columns were loaded. Rows used in the analysis: 149.

Class	Rows	Share
setosa	50	33.6%
versicolor	50	33.6%
virginica	49	32.9%

Table 1: Classes of *Species*.

Predictor columns: sepal_length, sepal_width, petal_length, petal_width.

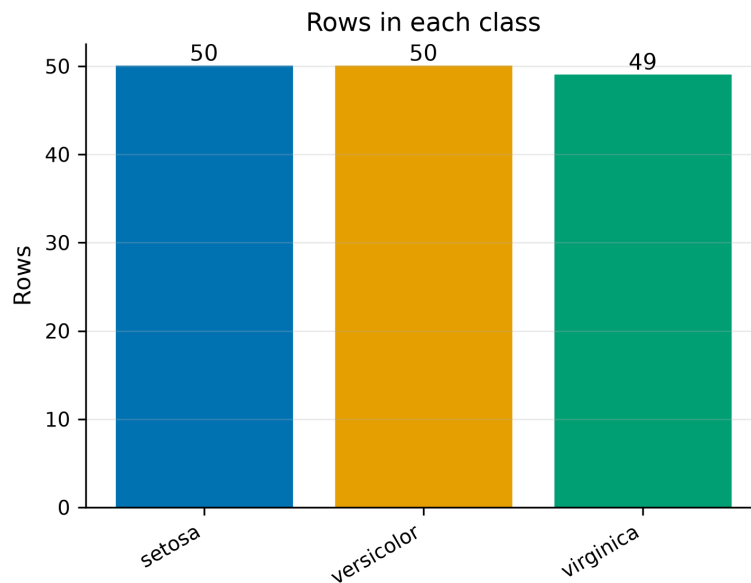


Figure 1: Number of rows in each class. Very unequal classes make some measures (e.g. accuracy) look better than they are; balanced accuracy takes this into account.

2 Methods (ready to adapt for your paper)

The paragraph below describes exactly what was done. You may copy and adapt it for the methods section of a paper or thesis.

Classification was carried out with EasyClassifier version 0.8.2 [1], which is built on scikit-learn [2], NumPy [3] and pandas [4]. The dataset (demo (Iris flowers)) contained 150 rows and 5 columns. The outcome to predict was *Species* with 3 classes. 1 duplicate row(s) were removed. The analysis used 149 rows and 4 predictor columns. Before modelling, numeric columns were standardised (mean 0, standard deviation 1) for Support Vector Machine (SVM), Logistic Regression and Neural Network (MLP), and scaled to the range 0–1 for KNN (Hassanat distance, k=5). All of these steps were learned from the training data only, separately within every validation split, so that no information from test data entered model training. The following classifiers were used with their default parameters, without hyperparameter tuning: Decision Tree [5], Random Forest [6], Support Vector Machine (SVM) [7, 8], Logistic Regression [9], KNN (Hassanat distance, k=5) [10], Naive Bayes [11] and Neural Network (MLP) [12, 13]. K-nearest neighbours used k = 5 and the Hassanat distance, applied in the standard form of the formula (all values non-negative) [14, 15]. Performance was estimated with 5-fold cross validation [16]. The classifier with the highest balanced accuracy was selected. To avoid optimistic bias from this selection, performance was estimated with nested 5-fold cross-validation, in which the complete comparison was repeated within each outer training fold and the winner was evaluated on the outer test fold [17, 18]. The contribution of each predictor was estimated by permutation importance [6], i.e. the drop in balanced accuracy when that column's values were randomly shuffled, measured on the test part of each cross-validation fold. Classifiers were ranked by balanced accuracy [19]; the measures reported were Accuracy, Precision, Recall, F1 and ROC AUC [20]. Figures were drawn with Matplotlib [21], including ROC curves [20].

3 Results

Final result for Neural Network (MLP) (measured on nested cross-validation) – report these numbers

Support Vector Machine (SVM) chosen in 2/5 folds; Neural Network (MLP) chosen in 1/5 folds; Decision Tree chosen in 1/5 folds; Naive Bayes chosen in 1/5 folds.

Measure	Value	What it means
Accuracy	95.30%	Accuracy is the share of predictions the model got right, out of all predictions.
Precision	95.29%	Precision measures how many of the items the model labeled positive are actually positive.
Recall	95.29%	Recall (sensitivity) measures how many of the real positive items the model successfully found.
F1	95.29%	F1 is a single score that balances precision and recall. It is high only when both are high.
ROC AUC	0.9633	ROC AUC measures how well the model separates classes across all thresholds. 1.0 is perfect, 0.5 is random guessing.

Comparison of classifiers

These scores were used only to choose the best classifier. They are slightly optimistic, because the winner partly won by luck; report the final result above instead.

Classifier	Balanced accuracy	Accuracy	F1	ROC AUC
Neural Network (MLP)	96.65%	96.64%	96.63%	0.9976
Logistic Regression	96.64%	96.64%	96.63%	0.9978
Support Vector Machine (SVM)	95.96%	95.97%	95.96%	0.9972
Decision Tree	95.29%	95.30%	95.29%	0.9647
KNN (Hassanat distance, k=5)	94.61%	94.63%	94.61%	0.9949
Random Forest	94.60%	94.63%	94.61%	0.9935
Naive Bayes	93.95%	93.96%	93.94%	0.9936

Table 2: Classifiers ranked by balanced accuracy.

Neural Network (MLP) and Logistic Regression had the same balanced accuracy; Neural Network (MLP) was selected because it is listed first. They perform equally well on this data.

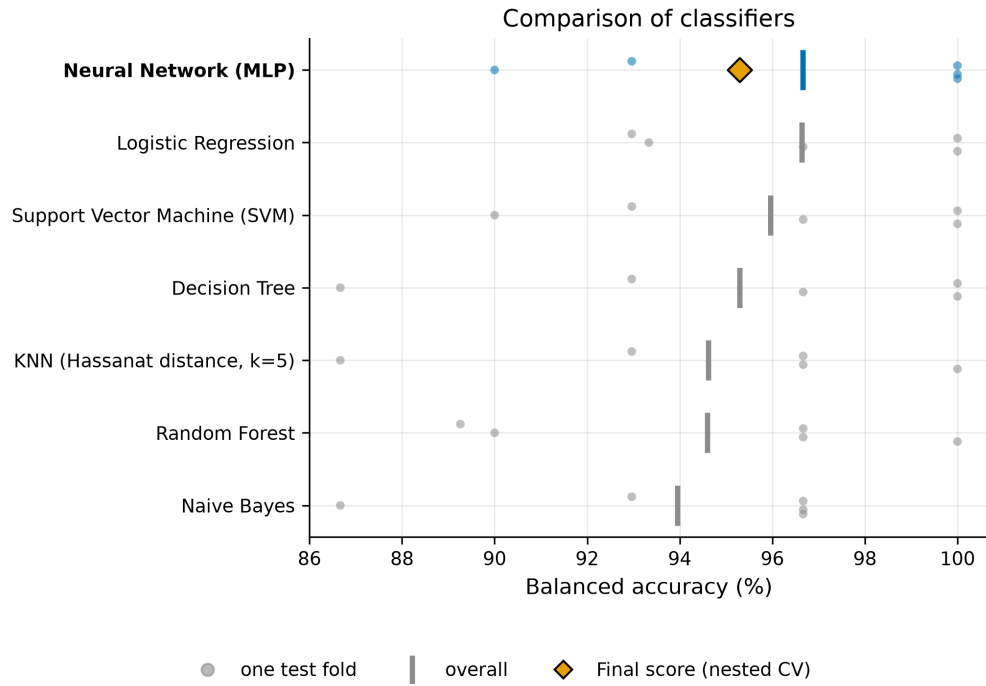


Figure 2: Balanced accuracy of each classifier. Dots are the individual test folds, the vertical bar is the overall score. The wider the dots are spread, the less certain the ranking. The diamond, if shown, is the final score of the selected classifier on data not used to choose it.

Figures for Neural Network (MLP) (final result)

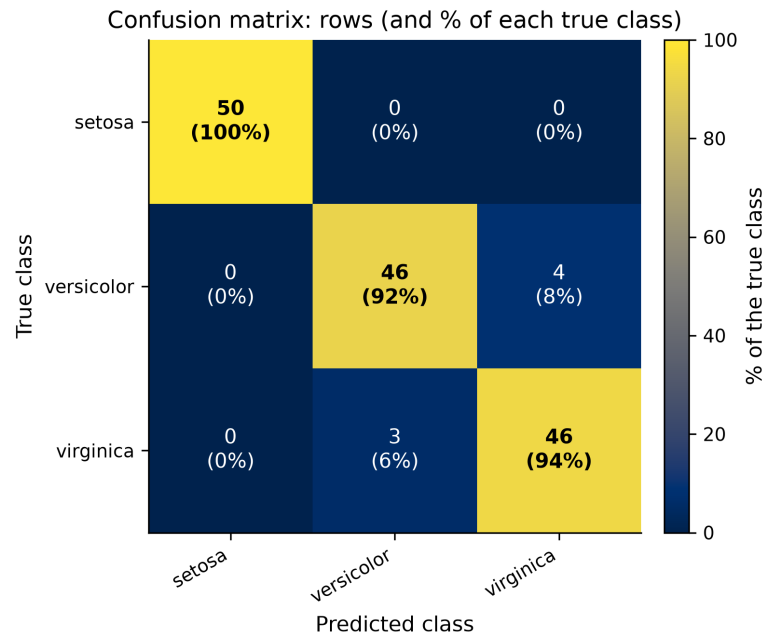


Figure 3: Confusion matrix. Rows are the true classes, columns the predicted classes; each cell shows the number of rows and the percentage of that true class. The diagonal holds the correct predictions; off-diagonal cells show which classes are confused.

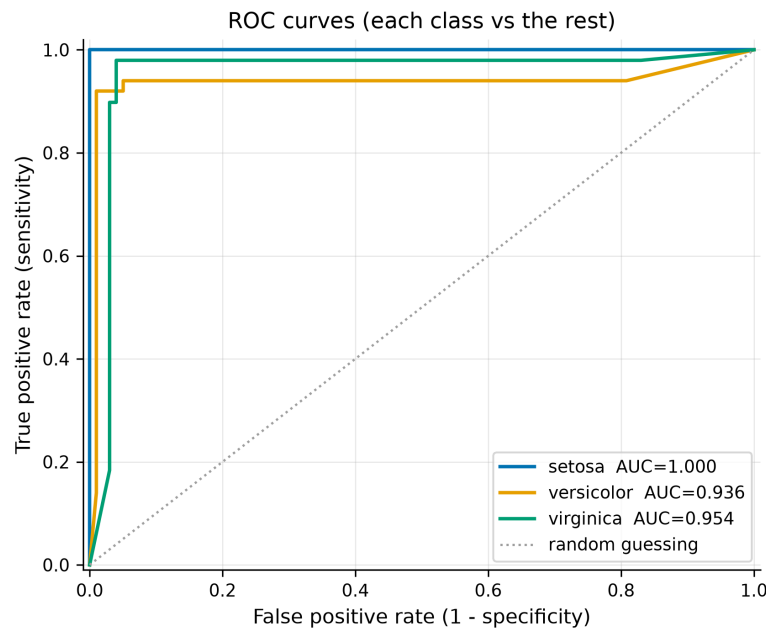


Figure 4: ROC curve(s). The closer a curve comes to the top-left corner, the better that class is separated from the others; the dotted diagonal is random guessing. AUC summarises each curve (1 = perfect, 0.5 = random).

Which columns mattered

Each column's values were shuffled, and the drop in balanced accuracy was measured on rows the model had not been trained on. A larger drop means the model relies more on that column. Values

near zero (or below) mean the column did not help. Importance shows what the model uses; it does not prove cause and effect.

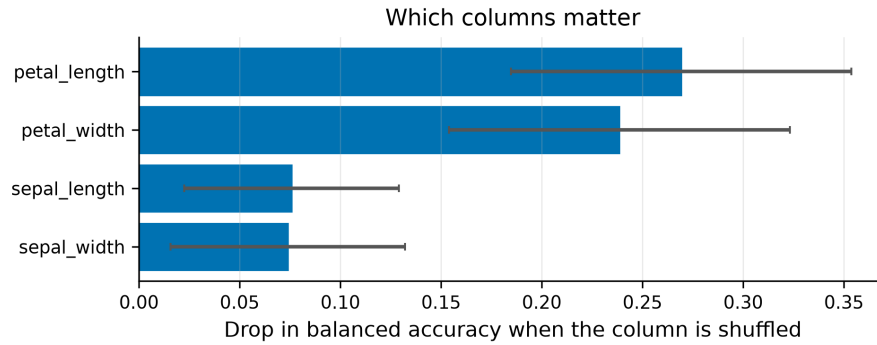


Figure 5: Permutation importance: how much the balanced accuracy drops when a column is shuffled (mean and spread over repeated shuffles).

Column	Importance	Spread
petal_length	0.2692	0.0844
petal_width	0.2385	0.0846
sepal_length	0.0757	0.0533
sepal_width	0.0739	0.0582

Table 3: Importance of each column.

4 Good practice when reporting

- Report the final result, not the best score from the comparison table.
- Mention the number of rows and the class sizes; results for small classes are less reliable.
- Classifiers used default parameters (no tuning), so the results can be reproduced exactly with the same data and EasyClassifier version.
- All choices and steps are recorded in `log.txt` in the Results folder.

References

- [1] Hassanat, A. B. A. (2026). EasyClassifier: Machine Learning without Programming (Version 0.8.2) [Software]. Zenodo. <https://doi.org/10.5281/zenodo.23122901>
- [2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [3] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- [4] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56-61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- [5] Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.

- [6] Breiman, L. (2001). Random Forests. *Machine Learning*, 45, 5-32. <https://doi.org/10.1023/A:1010933404324>
- [7] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297. <https://doi.org/10.1007/BF00994018>
- [8] Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), *Advances in Large Margin Classifiers* (pp. 61-74). MIT Press.
- [9] Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2), 215-242. <https://doi.org/10.1111/j.2517-6161.1958.tb00292.x>
- [10] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21-27. <https://doi.org/10.1109/TIT.1967.1053964>
- [11] Hand, D. J., & Yu, K. (2001). Idiot's Bayes - not so stupid after all? *International Statistical Review*, 69(3), 385-398. <https://doi.org/10.1111/j.1751-5823.2001.tb00465.x>
- [12] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. <https://doi.org/10.1038/323533a0>
- [13] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.
- [14] Hassanat, A. B., Alkafaween, E., Tarawneh, A. S., & Elmougy, S. (2022). Applications review of Hassanat distance metric. In *2022 International Conference on Emerging Trends in Computing and Engineering Applications (ETCEA)*, Karak, Jordan (pp. 1-6). IEEE. <https://doi.org/10.1109/ETCEA57049.2022.10009844>
- [15] Hassanat, A. B. (2014). Dimensionality invariant similarity measure. arXiv preprint arXiv:1409.0923. <https://arxiv.org/abs/1409.0923>
- [16] Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* (pp. 1137-1143).
- [17] Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7, 91. <https://doi.org/10.1186/1471-2105-7-91>
- [18] Cawley, G. C., & Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11, 2079-2107.
- [19] Brodersen, K. H., Ong, C. S., Stephan, K. E., & Buhmann, J. M. (2010). The balanced accuracy and its posterior distribution. In *20th International Conference on Pattern Recognition* (pp. 3121-3124). IEEE. <https://doi.org/10.1109/ICPR.2010.764>
- [20] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861-874. <https://doi.org/10.1016/j.patrec.2005.10.010>
- [21] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>